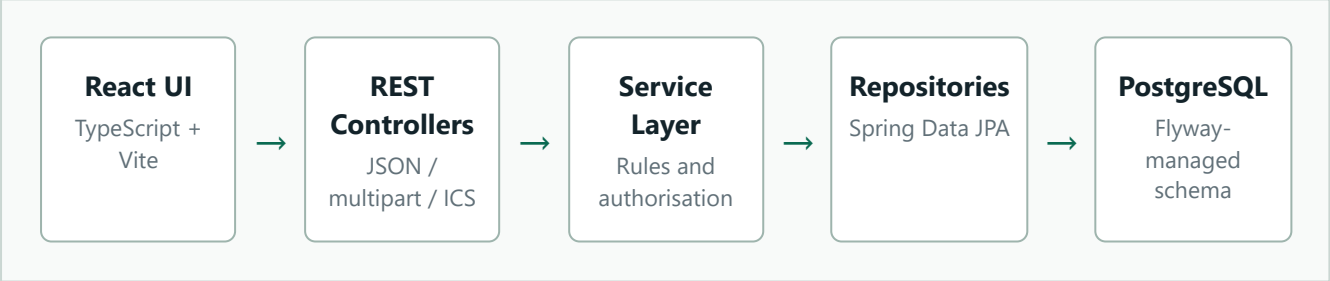
ACADEMIC WORKSPACE PLATFORM

One system for courses, deadlines and academic work.

UniHub is a full-stack web application that centralises information normally scattered across learning platforms, email, PDF files and messaging groups. It provides separate, authorised workflows for students, professors and administrators.

> **Project objective.** Demonstrate production-oriented full-stack fundamentals: relational modelling, REST API design, JWT authentication, role-based authorisation, file handling, scheduling, database migrations and a typed React user interface.

System architecture



3
authorised roles
Student, Professor and Administrator.

16
seeded assignments
Realistic deadlines, descriptions and points.

11
automated tests
Service, security and repository coverage.

Core capabilities

- Institutional-email registration and BCrypt password storage
- Course catalogue, search, filters and enrolment
- Assignments, completion progress and priority classification
- PDF material upload and authenticated download
- Weekly teaching/student schedules and ICS export
- Prerequisites, reviews and course recommendations
- In-app notifications and scheduled email reminders
- Professor ownership checks and administrator controls

Technology stack and request lifecycle

Frontend	**React 18 + TypeScript 5 + Vite 6.** Component state, typed API contracts, responsive layouts and Lucide icons. The browser stores the signed session token and sends it in the Authorization header.
API	**Java 17 + Spring Boot 3.3.** Spring MVC exposes REST endpoints; Jakarta Validation rejects malformed requests before business logic runs.
Security	**Spring Security + JWT + BCrypt.** Stateless authentication, role authorities, token expiry and endpoint/service ownership checks.
Persistence	**PostgreSQL + Spring Data JPA + Hibernate.** Entity relationships map to explicit foreign keys and uniqueness constraints.
Delivery	**Flyway + Maven + JUnit 5 + Mockito + H2.** Repeatable schema evolution and automated backend verification.
Email	**Spring Mail + scheduled jobs + Mailpit.** Local SMTP capture for development; configurable SMTP for production delivery.

Authenticated GET request

1. **React calls the API.** The typed wrapper builds a URL such as `/api/courses` and attaches `Authorization: Bearer <JWT>`.
2. **JwtAuthenticationFilter validates identity.** It extracts the email subject, loads current authorities and populates Spring's security context.
3. **The controller maps HTTP to an application operation.** DTOs define input/output boundaries; JPA entities are not exposed directly.
4. **The service executes rules.** It verifies role, course ownership, enrolment, capacity, prerequisites or review eligibility as required.
5. **The repository issues SQL.** Hibernate maps records to entities inside a transaction and the response is serialised as JSON.
6. **React updates state.** Loading, success, empty and error states are rendered without a full page reload.

Security behaviour

401 UNAUTHORISED

Missing or expired token. The frontend clears the stale session and returns to sign-in.

403 FORBIDDEN

Identity is valid, but the role or ownership rule does not permit the requested action.

PostgreSQL schema and domain relationships

The schema is managed by ten ordered Flyway migrations. Hibernate uses `ddl-auto: validate`, so application startup confirms that Java mappings match the database rather than silently altering production data.

users
Identity, institutional email, BCrypt password, role

courses
Professor FK, ECTS, faculty, department, capacity

enrollments
Student FK + course FK + enrolled timestamp

assignments
Course FK, deadline, points and description

assignment_completions
Per-student completion state and timestamp

materials
PDF metadata and generated storage filename

course_schedules
Weekday, start/end time and room

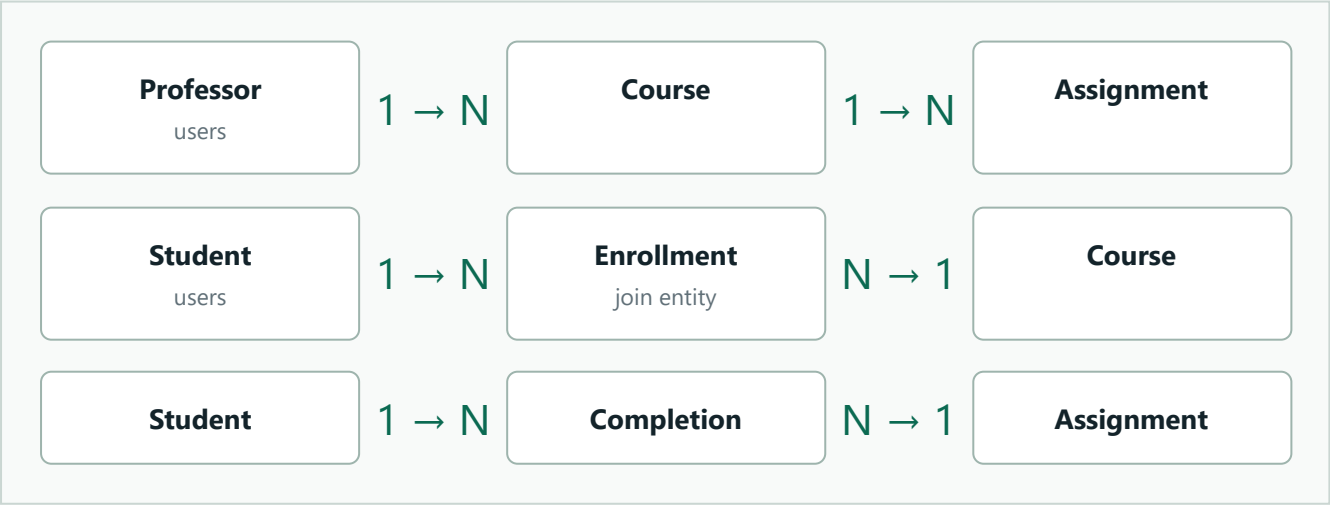
course_prerequisites
Self-referencing many-to-many course relation

course_reviews
Student/course uniqueness, rating and comment

notifications
User, type, reference key, read state

email_reminders
Assignment/user uniqueness prevents duplicates

Relationship diagram



Integrity rules

Constraint	Purpose
Unique student + course enrolment	Prevents a student from enrolling twice in the same course.
Unique student + assignment completion	Represents completion as a stable state rather than duplicate events.

S

Student role

Discovery, planning, progress and academic resources

Account and course discovery

- Registration accepts university-provided addresses while blocking common personal providers.
- The catalogue supports live search by course, professor or department, plus department filters and sorting.
- Enrolment checks capacity, duplicate membership and required prerequisite courses.
- Course detail exposes professor, ECTS, schedule, materials, assignments and reviews.

Daily academic workflow

- The dashboard aggregates enrolled courses and upcoming deadlines.
- Students mark assignments complete or undo completion.
- Progress is calculated as completed assignments divided by total assignments for each course.
- The weekly schedule displays only courses in which the student is enrolled.

Deadline Intelligence

Priority	Rule	Student meaning
URGENT	Less than 24 hours remaining	Immediate attention is required.
SOON	Between 24 and 72 hours	Plan work during the next study session.
OK	More than 72 hours	Visible in the plan but not yet critical.

Materials, calendar and communication

Materials
Authenticated PDF download preserves the original filename while files are stored under generated safe names.

Calendar
Assignments are exported as an `.ics` file that can be imported into Outlook, Google Calendar or another calendar application.

Notifications
New assignments, course materials and approaching deadlines create deduplicated in-app updates.

Reviews and recommendations

Reviews require an eligible enrolment period and are unique per student/course. Recommendations use a SQL co-enrolment query: courses frequently taken by students with similar enrolments are ranked without introducing unnecessary machine learning infrastructure.

> **Student boundary:** students cannot create courses, upload materials, publish schedules, create assignments, inspect another course's student list or change account roles.

P

Professor

Own and operate courses

A

Administrator

Govern users and catalogue

Professor workspace

- Create a course with name, description, faculty, department, ECTS credits and capacity.
- View only owned courses in the professor workspace, including total enrolment and published schedule metrics.
- Publish assignments through the course interface with title, description, European date/time and maximum points. The assignment appears immediately and enrolled students receive an internal notification.
- Upload validated PDF materials, define prerequisite courses and publish weekday/time/room schedule entries.
- Inspect the enrolled-student list and send a Mailpit test notification only to students enrolled in the owned course.

Administrator workspace

- List user accounts and promote authorised users between Student, Professor and Administrator roles.
- Create and manage courses across the full catalogue, independent of professor ownership.
- Create assignments, schedules, prerequisites and course resources for any course.
- Delete courses while database constraints remove dependent resources safely.

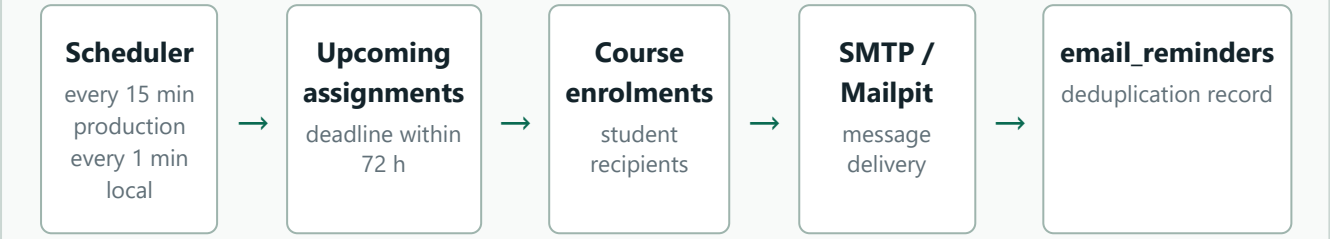
Permission matrix

Operation	Student	Professor	Administrator
Browse courses and details	**Allowed**	**Allowed**	**Allowed**
Enrol in a course	**Allowed**	Denied	Denied
Create a course	Denied	**Allowed**	**Allowed**
Create assignment/material/schedule	Denied	**Owned course**	**Any course**
View enrolled students	Denied	**Owned course**	**Any course**
Change user roles	Denied	Denied	**Allowed**
Delete a course	Denied	Denied	**Allowed**

> **Defence in depth.** Hiding a button in React improves usability, but it is not security. Every sensitive operation is verified again in the backend service using the current database user, role and course ownership.

Notifications, testing and local operation

Email reminder pipeline



Mailpit development mode: messages are accepted on SMTP port `1025` and displayed at `http://127.0.0.1:8025`. They remain local and never reach a real mailbox. Production delivery replaces Mailpit with a verified SMTP provider and environment-based credentials.

Quality and security controls

- JUnit 5 and Mockito service tests
- H2 repository integration test
- JWT generation/validation test
- Institutional email policy test
- Email sending and deduplication tests
- 10 MB multipart size limit
- PDF MIME, filename and magic-byte validation
- UUID storage names and path normalisation
- DTO validation and consistent HTTP errors
- Flyway migrations and foreign-key constraints

API surface and operational boundaries

Area	Representative endpoint	Responsibility
Identity	`POST /api/auth/login`	Authenticates credentials and returns a signed JWT.
Catalogue	`GET /api/courses`	Searches, filters and sorts the academic catalogue.
Teaching	`POST /api/courses/{id}/assignments`	Publishes assignments after role and ownership validation.
Student progress	`PUT /api/assignments/{id}/completion`	Records per-student completion state.
Calendar	`GET /api/calendar/export.ics`	Produces an interoperable academic calendar file.
Administration	`PATCH /api/admin/users/{id}/role`	Changes account roles under administrator authority.

Deployment model

Browser client
Static React assets can be hosted by a web server or frontend platform and configured through `VITE_API_URL`.

Application service
The Spring Boot process remains stateless; horizontal instances can share PostgreSQL and external file storage.

Managed services
PostgreSQL provides durable data, while object storage and verified SMTP replace local development adapters.